The `StaticFileMiddleware` in figure 6.4 is placed before the `RoutingMiddleware`, so it executes before an endpoint is selected. Conversely, the `AuthorizationMiddleware` is placed after the `RoutingMiddleware`, so it can tell which minimal API endpoint will be executed eventually. In addition, it can access certain metadata about the endpoint, such as its name and the permissions required to access it.

TIP The `AuthorizationMiddleware` needs to know which endpoint will be executed, so it must be placed after the `RoutingMiddleware` and before the `EndpointMiddleware` in your middleware pipeline. I discuss authorization in more detail in chapter 24.

It's important to remember the different roles of the two types of routing middleware when building your application. If you have a piece of middleware that needs to know which endpoint (if any) a given request will execute, you need to make sure to place it after the `RoutingMiddleware` and before the `EndpointMiddleware`.

TIP If you want to place middleware before the `RoutingMiddleware`, such as the `StaticFileMiddleware` in figure 6.4, you need to override the automatic middleware added by `WebApplication` by calling `UseRouting()` at the appropriate point in your middleware pipeline. See listing 4.3 in chapter 4 for an example.

I've covered how the `RoutingMiddleware` and `EndpointMiddleware` interact to provide routing capabilities in ASP.NET Core, but we've looked at only simple route templates so far. In